Un grand festin

Ce TP est à réaliser en C

Préliminaires

Vous êtes chargés d'organiser un grand pique-nique pour la classe des MPI. Vous avez eu une super idée, réaliser une grande salade composée ! Malheureusement vos camarades sont difficiles à satisfaire. Afin de rendre tout le monde aussi heureux que possible, vous avez demandé à chacun ses souhaits. Ils peuvent faire deux types de souhaits :

- Une demande de présence d'un aliment spécifique.
- Une demande d'absence d'un aliment spécifique.

Puisque vous êtes déjà bien gentil d'organiser un tel festin, vous limitez à deux le nombre de souhaits de chaque élève. Votre objectif est d'en réaliser au moins un pour chaque élève. Grâce à votre connaissance du cours d'informatique, vous pensez immédiatement à modéliser cette situation sous forme d'une formule 2-FNC.

FIGURE 1 – Si seulement vos camarades étaient aussi simple à satisfaire que Tchoupi…

Choix de la représentation

On décide de représenter une formule 2-FNC de la façon suivante :

```c
typedef struct clause
{
    int var1;
    int var2;
    struct clause* next;
} clause_t;

typedef struct formule
{
    int nombre_variables;
    int nombre_clauses;
    clause_t* expression;
} formule_t;
```

Ici, les formules sont des listes chaînées de clauses.

On représentera les variables par un entier qui leur est unique. Elles sont numérotées de 1 à n où n est le nombre de variables. Leur négation sera représenté par l'opposé de ce nombre. Par exemple, la formule $\varphi = (x_1 \vee x_2) \wedge (\neg x_2 \vee x_3)$ sera représentée par :

```
{nombre_variables = 3; nombre_clauses = 2; expression = [(1,2); (-2, 3)]}
```

On définit de plus une valuation comme un tableau de booléens de taille le nombre de variables de la formule. La variable x_i est valuée à vrai si et seulement si la case $i-1$ vaut `true`.

Question 1 : Écrire une fonction `int satisfait(formule_t phi, bool* valuation)` qui, étant données une formule et une valuation, teste si celle-ci satisfait la formule.

On énumère les valuations dans l'ordre lexicographique. Par exemple, si on a seulement trois variables, l'énumération sera :

- F, F, F
- F, F, V
- F, V, F
- F, V, V
- V, F, F
- V, F, V
- V, V, F
- V, V, V

Question 2 : Écrire une fonction `void valuation_suivante(bool* valuation, int n)` qui, étant donnée une valuation, calcule sa suivante dans l'ordre lexicographique.

Question 3 : Écrire une fonction `int est_satisfiable(formule_t phi)` qui, étant donnée une formule 2-FNC, teste si elle est satisfiable.

Algorithme de Kosaraju

On définit un graphe avec des listes d'adjacence :

```
typedef struct voisins
{
    int sommet;
    struct voisins* suivant;
}voisins_t;

typedef struct graphe
{
    int nombre_sommets;
    voisins_t** tableau;
}graphe_t;
```

On utilise ici encore des listes chaînées pour manipuler les voisins d'un sommet. La case i de `tableau` est un pointeur vers le premier chaînon de la liste des voisins si celle-ci est non vide.

Question 4 : Écrire une fonction `parcours_profondeur(graphe_t g, int n, int s, bool* deja_vus, int* ordre, int* adresse_indice)` qui, étant donnés un graphe g à n sommets s, un sommet s, un tableau des sommets déjà traités et un tableau `ordre` réalise un parcours en profondeur depuis s en respectant `deja_vus`. Le tableau `ordre` sera mis à jour à partir de la case indiquée par la valeur se trouvant à l'adresse `adresse_indice`. La consigne n'est pas forcément claire, demande de l'aide si vous avez du mal à la comprendre.

Question 5 : Écrire une fonction `int* ordre_suffixe(graphe_t g, int n)` qui, étant donné un graphe, renvoie un ordre suffixe obtenu après un parcours en profondeur.

Question 6 : Écrire une fonction `graphe transpose(graphe_t g, int n)` qui, étant donné un graphe, renvoie le graphe transposé associé.

Question 7 : Écrire une fonction `int* kosaraju(graphe_t g, int n)` qui, étant donné un graphe, renvoie le tableau des composantes fortement connexes. On numérotera les composantes comme on le souhaite.

Résoudre 2-SAT

Ces questions sont moins guidées, sortez une feuille, un crayon et munissez vous de votre imagination pour en venir à bout.

Question 8 : Écrire une fonction `int est_satisfiable_rapide(formule_t phi)` qui teste si une formule 2-SAT est satisfiable en utilisant les composantes fortement connexes.

Question 9 : Écrire une fonction `bool* valuation_rapide(formule_t phi)` qui, étant donnée une formule qu'on suppose satisfiable, renvoie une valuation qui la satisfait.